

6

Diving into CQRS

If you are going to build big applications, separating the read and write operations of a data source can help the application scale up quickly in the future. The **Command and Query Responsibility Segregation (CQRS)** pattern will help you write maintainable and testable code. The pattern would also be an ideal candidate for how to separate all the requests for reading and all the requests for modifying data.

This chapter is about the CQRS pattern, the mediator pattern, and the popular MediatR NuGet package for CQRS and pipeline behavior.

We will cover the following topics:

- What is CQRS?
- What is the mediator pattern?
- What is the MediatR package?
- Why learn CQRS?
- When to use CQRS
- Drawbacks of CQRS

Technical requirements

You will need Visual Studio 2019, Visual Studio for Mac, or Rider to complete this chapter.

What is CQRS?

CQRS stands for **Command and Query Responsibility Segregation**. What this means is that commands and queries should have separate responsibilities and clear domain boundaries.

Let say you have a controller, and the controller has a few endpoints for getting data, creating data, updating data, and removing data, and these are your *GET*, *POST*, *PUT*, and *DELETE* methods in short.

Everything in the controller that gets data or does not mutate data falls under *Query*; while everything else that mutates data, such as *POST*, *PUT*, and *DELETE* requests, is classified as *Command*.

Now, your application should have a query model that handles the query for getting data from the database. It should also have a command model that handles the command for writing or deleting data in the database.

The following is a diagram of a command that goes to your application:

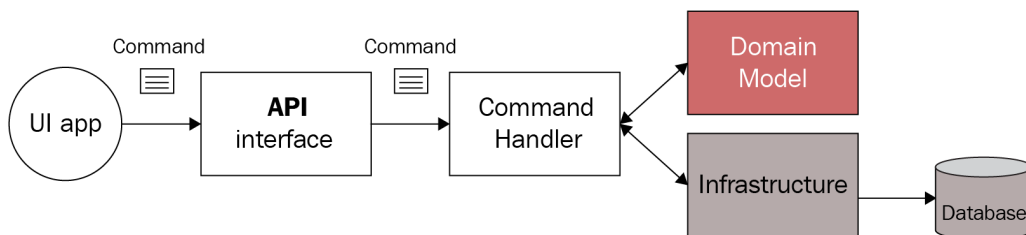


Figure 6.1 – An app sending an HTTP request

The preceding *Figure 6.1* shows the UI sends a *POST/UPDATE/DELETE* request to an API endpoint, passing through the command handler. After the command handler, the request goes to the domain model and infrastructure to modify the database.

The segregation of queries and commands is the separation we are after. However, a mediator, which is a pattern, will help us to decouple everything.

Now, let's see another example where we can see a controller:

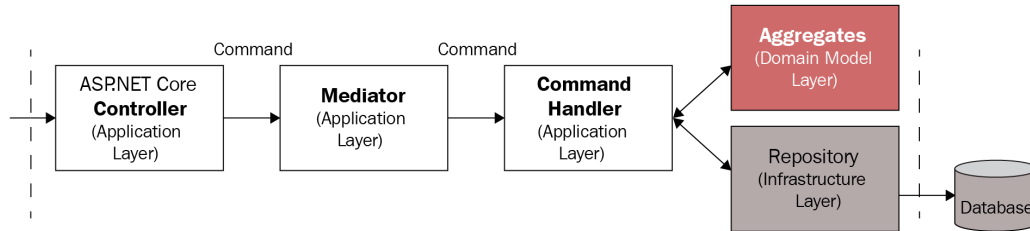


Figure 6.2 – A controller sending a command

In *Figure 6.2*, the ultimate goal is to bring a GET request and have a post, like in the preceding figure, to end up in a mediator. This mediator will find a query handler and a command handler to handle the requests, and through that, the controller won't need to know if any service is injected into handlers.

The design makes the application more manageable and way more testable because you can unit test the handlers isolated from the whole controller. After all, there is nothing in the controller. The controller just has a query sent to the mediator and eventually lands in a handler, but it's a very decoupled approach and remarkably predictable for your application.

So that is CQRS. Now, let's move on to the next section to find out what the mediator pattern is.

What is the mediator pattern?

Suppose you have four services, objects, or elements in your application, and these services, objects, or elements need to communicate generally to each other. *ServiceA* needs to talk to *ServiceB* and *ServiceD*, and *ServiceC* needs to talk to *ServiceB* and *ServiceD*, while *ServiceB* also needs to talk to *ServiceD*. These services, objects, and elements are now tightly coupled with each other.

Here comes the Mediator to the rescue. Instead of services, objects, or elements calling each other, we will put a mediator in the middle. The mediator acts like an airport traffic control tower that knows how we want our services to communicate with each other, as shown in the following figure:

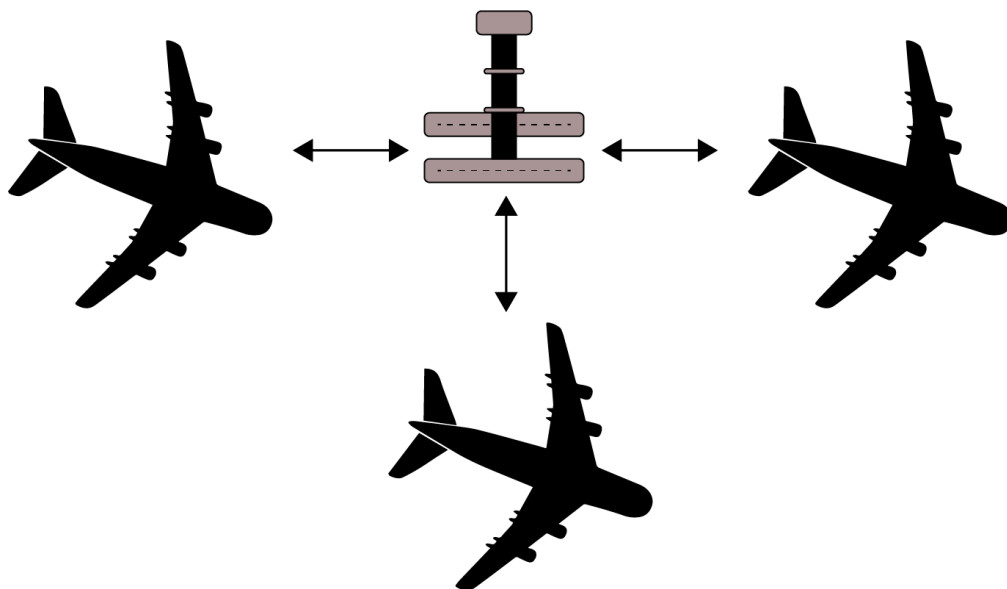


Figure 6.3 – Mediator as a traffic control tower

The mediator in *Figure 6.3* is the traffic control tower that handles the communication between the services, objects, or elements (the airplanes in this case).

So, how do you do or implement the mediator pattern in ASP.NET Core? Let's find out in the next section.

What is the MediatR package?

So, what does the NuGet MediatR package do? The MediatR package is an implementation of the mediator pattern in .NET that is ready to use.

The MediatR package is not complicated to use. You only need to remember that one request has a respective handler, which returns one response, as shown in the following figure:

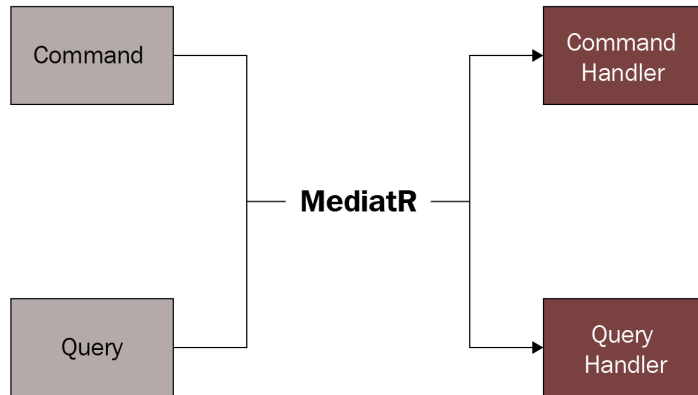


Figure 6.4 – MediatR is an implementation of the mediator pattern

You can see in *Figure 6.4* that you can have several requests. One request enters the MediatR package and is followed by another one. You have the mediator in the middle, which accepts the requests and finds the appropriate handlers that need to handle those requests. Each request has one handler; the handler does all the magic and returns the right response for the correct request.

The MediatR package is like a magic box where you push the request and you don't know downstream who will handle the request. Addressing the requests now becomes the responsibility of MediatR. Hence, a reason to use MediatR is that it completely decouples the controller from the application and the business logic.

So, that is the MediatR package, a package that you can use for CQRS. By why learn CQRS? Let's find out in the next section.

Why learn CQRS?

All your controllers will only have a single dependency, which is the MediatR package. Each one of your actions will just call a method, `Mediator.send`, and return a result, making your controllers slimmer than writing it without the CQRS pattern.

Okay, so what about when to use CQRS? Let's see in the next section.

When to use CQRS

Why would you want to use the mediator or the CQRS patterns? There are a lot of reasons for sure, but we will only go over the apparent reasons, which are as follows:

- **Services call each other:** All the read and write requests go into the box in the middle (the mediator) and then come out. If you want to trigger any request from anywhere, the request has to hit the box in the middle, the mediator.
- **Clean code in large projects:** The use of the mediator and mediator pipelines will help you shrink your controller sizes and move your business logic into their respective files. Hence, it will be effortless to traverse the folder structure and find the logic you're looking for.
- **A boundary between writes and reads:** Any UI team can efficiently implement a UI that requires more data from the database due to the separation of reads and writes.

A UI team can work freely without worrying about affecting the backend work and how logic may distribute notifications or write to some other channel to trigger further services.

- **Portability of the code:** The mediator can trigger services that you reused in a separate project. You can do this by bringing the mediator and importing the project with services and see that your services are accessible through this box.

Okay, so these are the obvious reasons to use CQRS; the next question is whether there are any cons to using CQRS. Of course, pros have cons in most cases. The cons are covered in the next section.

Drawbacks of CQRS

CQRS will bring extra code if you combine CQRS with the repository pattern. My suggestion is not to add another abstraction such as the repository pattern for the following reasons:

- Entity Framework already implements the repository pattern.
- There is a 99% chance that you will not change your Entity Framework implementation with NHibernate ORM or Dapper Micro ORM.

Okay, so we just finished exploring what CQRS, Mediator, and the MediatR package are. Let's summarize what you have learned here.

Summary

You have learned that CQRS separates commands, which are requests that mutate or write data, and queries, which are requests to read data. You also learned how CQRS helps you to write slimmer controllers.

You have also learned that the Mediator design pattern acts like an air traffic controller between commands/queries and the handlers. You learned the implementation of a mediator pattern that you can use in .NET, the MediatR NuGet package, a time saver because you don't need to implement it yourself, and using Mediator makes your code cleaner and more maintainable.

You also have seen drawbacks of using CQRS, including writing extra code, but the additional code means implementing CQRS to make cleaner and more maintainable code.

In the next chapter, we will apply the CQRS pattern and the mediator pattern and use the MediatR NuGet package in ASP.NET Core 5 to build a highly scalable and maintainable Web API.